

CSCI-4208: Developing Advanced Web Applications

Week 9: Lecture 15
Node.js (Basics)

Overview:

1. Node.js: Introduction
2. Node.js: Command-line
3. Node.js: REPL console
4. Node.js: Console output
5. Node.js: Console input
6. Node.js: Modules
7. Node.js: Event loop
8. Node.js: Event Emitter
9. Node.js: Asynchronous JS
10. Node.js: Streams
11. Node.js: Buffers
12. Node.js: Error Handling

1 Node.js: Introduction - What is Node.js?

- Node.js is an open source, cross-platform JavaScript runtime environment & is popular for developing server-side and networking applications. Node.js applications are written in JavaScript, and can be run on OS X, Microsoft Windows, and Linux.
- Node.js runs the V8 JavaScript engine, the core of Google Chrome, outside of the browser, allowing JavaScript for any kind of general-purpose coding project. Node.js uses an event-driven, non-blocking I/O model that makes it lightweight and efficient.
- Node.js also provides a rich library of various JavaScript modules which simplifies the development of web applications.

Node.js = Runtime Environment + JavaScript Library

1 Node.js: Introduction - What is Node.js?

- A Node.js app is run in a single process, without creating a new thread for every request. Node.js provides a set of asynchronous I/O primitives in its standard library that prevent JavaScript code from blocking and generally, libraries in Node.js are written using non-blocking paradigms, making blocking behavior the exception rather than the norm.
- When Node.js performs an I/O operation, like reading from the network, accessing a database or the filesystem, instead of blocking the thread and wasting CPU cycles waiting, Node.js will resume the operations when the response comes back.
- This allows Node.js to handle thousands of concurrent connections with a single server without introducing the burden of managing thread concurrency, which could be a significant source of bugs.

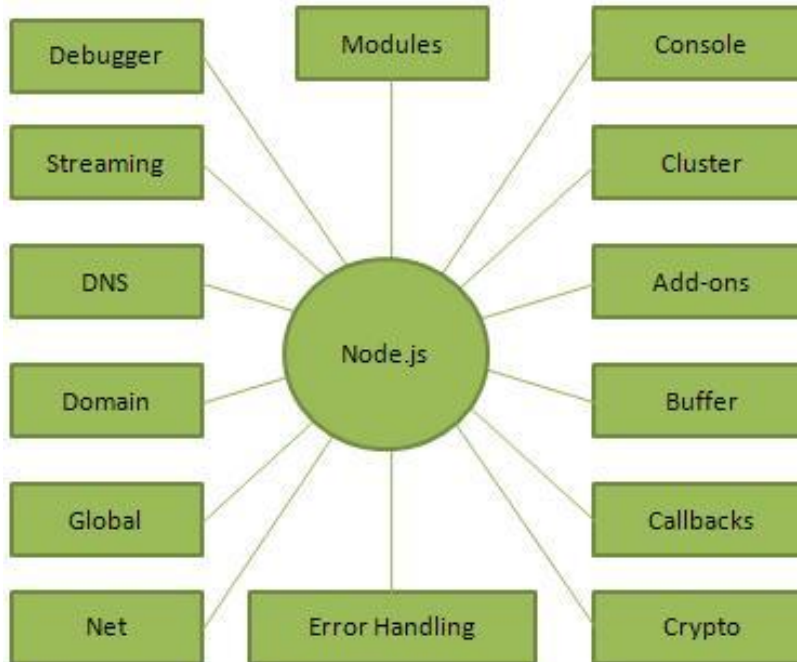
1 Node.js: Introduction - Features

Following are some of the important features that make Node.js the first choice of software architects.

- **Asynchronous and Event Driven** – All APIs of Node.js library are asynchronous, that is, non-blocking. It essentially means a Node.js based server never waits for an API to return data. The server moves to the next API after calling it and a notification mechanism of Events of Node.js helps the server to get a response from the previous API call.
- **Very Fast** – Being built on Google Chrome's V8 JavaScript Engine, Node.js library is very fast in code execution.
- **Single Threaded but Highly Scalable** – Node.js uses a single threaded model with event looping. Event mechanism helps the server to respond in a non-blocking way and makes the server highly scalable as opposed to traditional servers which create limited threads to handle requests. Node.js uses a single threaded program and the same program can provide service to a much larger number of requests than traditional servers like Apache HTTP Server.
- **No Buffering** – Node.js applications never buffer any data. These applications simply output the data in chunks.
- **License** – Node.js is released under the MIT license.

1 Node.js: Introduction - Concepts

This diagram depicts some important parts of Node.js



1 Node.js: Introduction - Standard library & Frameworks

Standard Library

Node.js has a fantastic standard library, including first-class support for networking.

<https://nodejs.org/api/>

Node.js Frameworks and Tools

Node.js is a low-level platform. In order to make things easy and exciting for developers, thousands of libraries were built upon Node.js by the community.

- ***Express***: It provides one of the most simple yet powerful ways to create a web server. Its minimalist approach, unopinionated, focused on the core features of a server, is key to its success.
- ***Socket.io***: A real-time communication engine to build network applications.

1 Node.js: Introduction - Comparison: Node.js & Browser

- Both the Browser and Node.js use JavaScript as their programming language.
- In the browser, most common tasks include interacting with the DOM, or other Web Platform APIs like Cookies. Those do not exist in Node.js. You don't have the document, window and all the other objects that are provided by the browser.
- In the browser, we don't have all the APIs that Node.js provides through its modules, like the filesystem access functionality.
- Another difference is that Node.js uses the CommonJS module system, while in the browser we see the ES Module standard. In practice, this means that you use `require()` statements in Node.js and `import` statements in the browser.

1 Node.js: Introduction - Comparison: Node.js & Browser

The V8 JavaScript Engine

- V8 is the name of the JavaScript engine that powers Google Chrome. It's the thing that takes our JavaScript and executes it while browsing with Chrome.
- V8 provides the runtime environment in which JavaScript executes. The DOM, and the other Web Platform APIs are provided by the browser.
- The V8 JavaScript engine is independent of the browser in which it's hosted. This key feature enabled the rise of Node.js.
- The Node.js ecosystem is huge and thanks to it V8 also powers desktop apps, with projects like Electron.

2 Node.js: Command-line - How to install Node.js

Official packages for all the major platforms are available at:

<https://nodejs.org/en/download/>

Downloads

Latest LTS Version: **14.16.0** (includes npm 6.14.11)

Download the Node.js source code or a pre-built installer for your platform, and start developing today.

LTS Recommended For Most Users	Current Latest Features	
 Windows Installer node-v14.16.0-x86.msi	 macOS Installer node-v14.16.0.pkg	 Source Code node-v14.16.0.tar.gz

Windows Installer (.msi)	32-bit	64-bit
Windows Binary (.zip)	32-bit	64-bit
macOS Installer (.pkg)	64-bit	
macOS Binary (.tar.gz)	64-bit	
Linux Binaries (x64)	64-bit	
Linux Binaries (ARM)	ARMv7	ARMv8
Source Code	node-v14.16.0.tar.gz	

2 Node.js: Command-line - Run JS scripts

To run a JavaScript program using Node.js use **node**, the globally available command (once you install Node.js), and pass the name of the file you want to execute.

Example:

If your main Node.js application file is `app.js`, you can call it by typing:

```
node app.js
```

While running the command, make sure you are in the same directory which contains the `app.js` file.

2 Node.js: Command-line - Run JS scripts with Arguments

You can pass any number of arguments when invoking a Node.js application using

```
node app.js arg1 arg2 argn
```

Arguments can be standalone or have a key and a value.

Examples:

```
node app.js joe
```

2 Node.js: Command-line - Run JS scripts with Arguments

To retrieve argument, use the built-in process object built which contains an argv property, which is an array that contains all the command line invocation arguments.

The process.argv Array:

- The first element is the full path of the node command.
- The second element is the full path of the file being executed.
- All the additional arguments are present from the third position going forward.

Demo:

```
process.argv.forEach((val, index) => {  
  console.log(`${index}: ${val}`)  
})
```

2 Node.js: Command-line - Launch REPL console

To run the Node.js REPL

```
node
```

REPL also known as Read Evaluate Print Loop is an interactive programming language environment (Basically a console window) that takes JavaScript expressions as user input and returns the result back to the console after execution.

3 Node.js: **REPL console** - *Introduction*

REPL stands for Read Eval Print Loop and is an interpreter console where JS commands are entered and the system responds with the resulting output in an interactive mode. Node.js comes bundled with a REPL environment. It performs the following tasks –

- Read – Reads user's input, parses the input into JavaScript data-structure, and stores in memory.
- Eval – Takes and evaluates the data structure.
- Print – Prints the result.
- Loop – Loops the above command until the user presses ctrl-c twice.

The REPL feature of Node is very useful in experimenting with Node.js codes and to debug JavaScript codes.

3 Node.js: **REPL console** - *Simple Expressions*

To simplify your learning, we have set up an easy to use Node.js REPL environment online, where you can practice Node.js syntax – Launch Node.js REPL Terminal

Starting REPL

REPL can be started by simply running node on shell/console without any arguments as follows.

```
node
```

Simple Expression

Let's try a simple mathematics at the Node.js REPL command prompt –

```
$ node
> 1 + 3
4
> 1 + ( 2 * 3 ) - 4
3
>
```


3 Node.js: **REPL console** - *Use Variables*

Use Variables

You can make use variables to store values and print later like any conventional script. If var keyword is not used, then the value is stored in the variable and printed. Whereas if var keyword is used, then the value is stored but not printed. You can print variables using console.log().

```
$ node
> x = 10
10
> var y = 10
undefined
> x + y
20
> console.log("Hello World")
Hello World
undefined
```

3 Node.js: **REPL console** - *Multiline Expressions*

Multiline Expression

Node REPL supports multiline expression similar to JavaScript. Let's check the following do-while loop in action –

```
$ node
> var x = 0
undefined
> do {
    ... x++;
    ... console.log("x: " + x);
    ... }
while ( x < 5 );
x: 1
x: 2
x: 3
x: 4
x: 5
undefined
>
```

... comes automatically when you press Enter after the opening bracket. Node automatically checks the continuity of expressions.

Node.js: **REPL console** - *Underscore Variable*

Underscore Variable

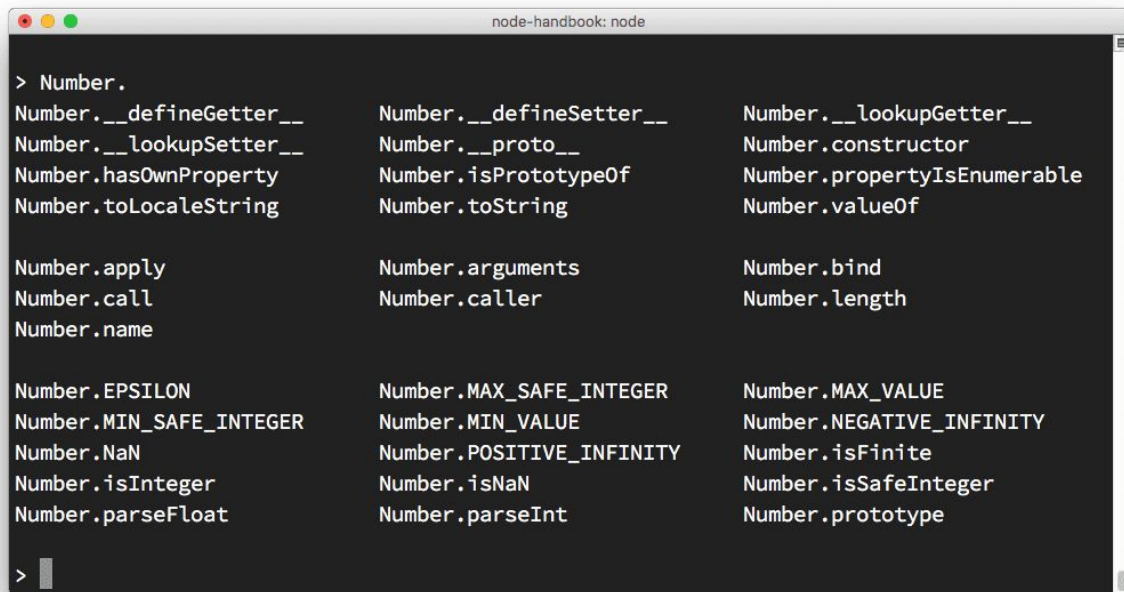
You can use underscore (`_`) to get the last result –

```
$ node
> var x = 10
undefined
> var y = 20
undefined
> x + y
30
> var sum = _
undefined
> console.log(sum)
30
undefined
>
```

3 Node.js: **REPL console** - *Inspect JS Objects*

Try entering the name of a JavaScript class, like `Number`, add a dot and press tab.

The REPL will print all the properties and methods you can access on that class:



```
node-handbook: node
> Number.
Number.__defineGetter__      Number.__defineSetter__    Number.__lookupGetter__
Number.__lookupSetter__     Number.__proto__           Number.constructor
Number.hasOwnProperty        Number.isPrototypeOf        Number.propertyIsEnumerable
Number.toLocaleString       Number.toString            Number.valueOf

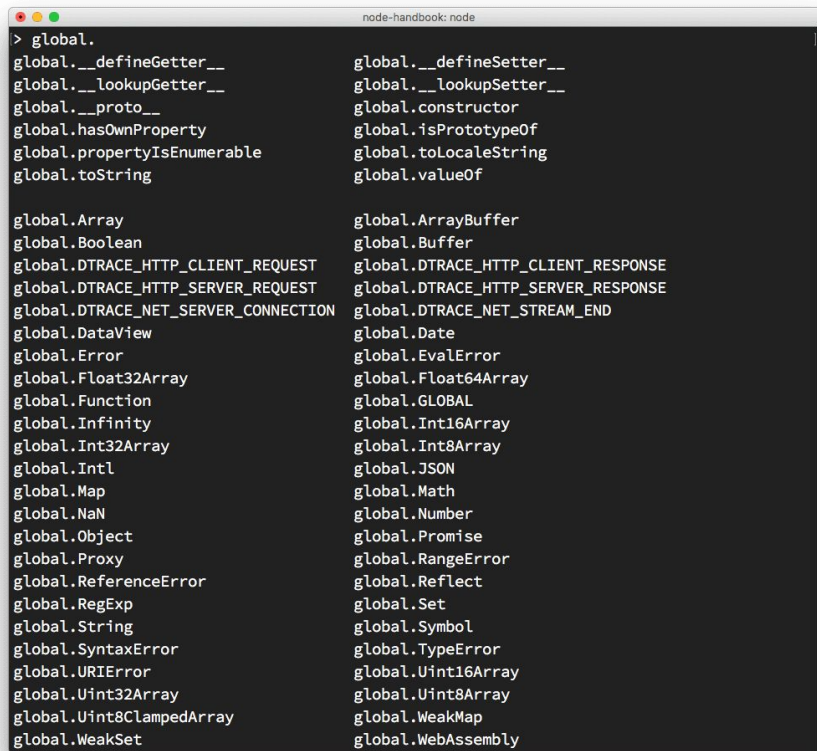
Number.apply                 Number.arguments           Number.bind
Number.call                  Number.caller              Number.length
Number.name

Number.EPSILON               Number.MAX_SAFE_INTEGER    Number.MAX_VALUE
Number.MIN_SAFE_INTEGER     Number.MIN_VALUE           Number.NEGATIVE_INFINITY
Number.NaN                   Number.POSITIVE_INFINITY   Number.isFinite
Number.isInteger             Number.isNaN                Number.isSafeInteger
Number.parseFloat            Number.parseInt             Number.prototype

> 
```

3 Node.js: **REPL console** - *Inspect global*

You can inspect the globals you have access to by typing `global.` and pressing tab:



```
node-handbook: node
> global.
global.__defineGetter__      global.__defineSetter__
global.__lookupGetter__     global.__lookupSetter__
global.__proto__            global.constructor
global.hasOwnProperty        global.isPrototypeOf
global.propertyIsEnumerable  global.toLocaleString
global.toString             global.valueOf

global.Array                 global.ArrayBuffer
global.Boolean               global.Buffer
global.DTRACE_HTTP_CLIENT_REQUEST  global.DTRACE_HTTP_CLIENT_RESPONSE
global.DTRACE_HTTP_SERVER_REQUEST  global.DTRACE_HTTP_SERVER_RESPONSE
global.DTRACE_NET_SERVER_CONNECTION  global.DTRACE_NET_STREAM_END
global.DataView              global.Date
global.Error                  global.EvalError
global.Float32Array          global.Float64Array
global.Function               global.GLOBAL
global.Infinity               global.Int16Array
global.Int32Array             global.Int8Array
global.Intl                   global.JSON
global.Map                    global.Math
global.NaN                    global.Number
global.Object                 global.Promise
global.Proxy                  global.RangeError
global.ReferenceError         global.Reflect
global.RegExp                 global.Set
global.String                 global.Symbol
global.SyntaxError            global.TypeError
global.URIError               global.Uint16Array
global.Uint32Array            global.Uint8Array
global.Uint8ClampedArray      global.WeakMap
global.WeakSet                global.WebAssembly
```

3 Node.js: **REPL console** - *Keyboard Shortcuts*

REPL Commands

- `ctrl + c` - terminate the current command.
- `ctrl + c` twice - terminate the Node REPL.
- `ctrl + d` - terminate the Node REPL.
- Up/Down Keys - see command history and modify previous commands.
- tab Keys - list of current commands.

3 Node.js: **REPL console** - *DOT commands*

REPL Commands

- `.help` - list of all commands.
- `.break` - exit from multiline expression.
- `.clear` - exit from multiline expression.
- `.save filename` - save the current Node REPL session to a file.
- `.load filename` - load file content in current Node REPL session.
- `.editor` - enables editor mode, to write multiline JavaScript code with ease.
Once you are in this mode, enter ctrl-D to run the code you wrote.
- `.exit` - exits the repl

3 Node.js: **REPL console** - *Stopping REPL*

Stopping REPL

As mentioned above, you will need to use ctrl-c twice to come out of Node.js REPL.

```
$ node  
>  
(^C again to quit)  
>
```


4 Node.js: Console output

Basic output using the console module

Node.js provides a console module which provides tons of very useful ways to interact with the command line.

It is basically the same as the console object you find in the browser.

The most basic and most used method is `console.log()`, which prints the string you pass to it to the console.

If you pass an object, it will render it as a string.

You can pass multiple variables to `console.log`, for example:

```
const x = 'x'  
const y = 'y'  
console.log(x, y)
```

4 Node.js: Console output

Clear the console

`console.clear()` clears the console (the behavior might depend on the console used)

Counting elements

`console.count()` is a handy method.

Print the stack trace

There might be cases where it's useful to print the call stack trace of a function, maybe to answer the question how did you reach that part of the code?

You can do so using `console.trace()`:

4 Node.js: Console output

Calculate the time spent

You can easily calculate how much time a function takes to run, using `time()` and `timeEnd()`

```
const doSomething = () => console.log('test')
const measureDoingSomething = () => {
  console.time('doSomething()')
  //do something, and measure the time it takes
  doSomething()
  console.timeEnd('doSomething()')
}
measureDoingSomething()
```

5 Node.js: Console input

Accept input from the command line in Node.js

How to make a Node.js CLI program interactive?

Node.js since version 7 provides the readline module to perform exactly this: get input from a readable stream such as the process.stdin stream, which during the execution of a Node.js program is the terminal input, one line at a time.

```
const readline = require('readline').createInterface({
  input: process.stdin,
  output: process.stdout
})

readline.question(`What's your name?`, name => {
  console.log(`Hi ${name}!`)
  readline.close()
})
```

5 Node.js: Console input

This piece of code asks the username, and once the text is entered and the user presses enter, we send a greeting.

The `question()` method shows the first parameter (a question) and waits for the user input. It calls the callback function once enter is pressed.

In this callback function, we close the readline interface.

`readline` offers several other methods, and I'll let you check them out on the package documentation linked above.

6 Node.js: Modules - Import syntax

Expose functionality from a Node file using exports

Node.js has a built-in module system.

A Node.js file can import functionality exposed by other Node.js files.

When you want to import something you use

```
const library = require('./library')
```

to import the functionality exposed in the library.js file that resides in the current file folder.

In this file, functionality must be exposed before it can be imported by other files.

Any other object or variable defined in the file by default is private and not exposed to the outer world.

6 Node.js: Modules - Import syntax

Expose functionality from a Node file using exports

This is what the `module.exports` API offered by the module system allows us to do.

When you assign an object or a function as a new `exports` property, that is the thing that's being exposed, and as such, it can be imported in other parts of your app, or in other apps as well.

You can do so in 2 ways.

The first is to assign an object to `module.exports`, which is an object provided out of the box by the module system, and this will make your file export just that object:

```
// car.js
const car = {
  brand: 'Ford',
  model: 'Fiesta'
}
module.exports = car
```

```
// index.js

const car = require('./car')
```

6 Node.js: Modules - Import syntax

Expose functionality from a Node file using exports

The second way is to add the exported object as a property of exports. This way allows you to export multiple objects, functions or data:

```
const car = {  
  brand: 'Ford',  
  model: 'Fiesta'  
}  
exports.car = car
```

or

```
exports.car = {  
  brand: 'Ford',  
  model: 'Fiesta'  
}
```

And in the other file, you'll use it by referencing a property of your import:

```
const items = require('./items')  
items.car
```

or

```
const car = require('./items').car
```

What's the difference between `module.exports` and `exports`?

The first exposes the object it points to. The latter exposes the properties of the object it points to.

7 Node.js: Event Loop - Introduction

Introduction

The Event Loop is one of the most important aspects to understand about Node.js. Because it explains how Node.js can be asynchronous and have non-blocking I/O..

The Node.js JavaScript code runs on a single thread. There is just one thing happening at a time. This is a limitation that's actually very helpful, as it simplifies a lot how you program without worrying about concurrency issues.

You just need to pay attention to how you write your code and avoid anything that could block the thread, like synchronous network calls or infinite loops.

In general, in most browsers there is an event loop for every browser tab, to make every process isolated and avoid a web page with infinite loops or heavy processing to block your entire browser.

The environment manages multiple concurrent event loops, to handle API calls for example. Web Workers run in their own event loop as well.

You mainly need to be concerned that your code will run on a single event loop, and write code with this thing in mind to avoid blocking it.

7 Node.js: Event Loop - Blocking

Blocking the event loop

Any JavaScript code that takes too long to return back control to the event loop will block the execution of any JavaScript code in the page, even block the UI thread, and the user cannot click around, scroll the page, and so on.

Almost all the I/O primitives in JavaScript are non-blocking. Network requests, filesystem operations, and so on. Being blocking is the exception, and this is why JavaScript is based so much on callbacks, and more recently on promises and `async/await`.

7 Node.js: Event Loop - the call stack

The call stack

The call stack is a LIFO (Last In, First Out) stack.

The event loop continuously checks the call stack to see if there's any function that needs to run.

While doing so, it adds any function call it finds to the call stack and executes each one in order.

You know the error stack trace you might be familiar with, in the debugger or in the browser console? The browser looks up the function names in the call stack to inform you which function originates the current call:

```
> const bar = () => {  
    throw new DOMException()  
}  
  
const baz = () => console.log('baz')  
  
const foo = () => {  
    console.log('foo')  
    bar()  
    baz()  
}  
  
foo()  
foo  
  
✖ ▼ Uncaught DOMException  
    bar      @ VM570:2  
    foo      @ VM570:9  
    (anonymous) @ VM570:13  
  
> |
```

7 Node.js: Event Loop - Queuing functions

Queuing function execution

Let's see how to defer a function until the stack is clear.

The use case of `setTimeout(() => {}, 0)` is to call a function, but execute it once every other function in the code has executed.

```
const bar = () => console.log('bar')
const baz = () => console.log('baz')

const foo = () => {
  console.log('foo')
  setTimeout(bar, 0)
  baz()
}

foo()
```

7 Node.js: Event Loop - Message Queue

The Message Queue

When `setTimeout()` is called, the Browser or Node.js starts the timer. Once the timer expires, in this case immediately as we put 0 as the timeout, the callback function is put in the Message Queue.

The Message Queue is also where user-initiated events like click or keyboard events, or fetch responses are queued before your code has the opportunity to react to them. Or also DOM events like `onLoad`.

The loop gives priority to the call stack, and it first processes everything it finds in the call stack, and once there's nothing in there, it goes to pick up things in the message queue.

We don't have to wait for functions like `setTimeout`, `fetch` or other things to do their own work, because they are provided by the browser, and they live on their own threads. For example, if you set the `setTimeout` timeout to 2 seconds, you don't have to wait 2 seconds - the wait happens elsewhere.

7 Node.js: Event Loop - Job Queue

Job Queue

Job Queue, which is used by Promises. It's a way to execute the result of an async function as soon as possible, rather than being put at the end of the call stack.

Promises that resolve before the current function ends will be executed right after the current function.

I find nice the analogy of a rollercoaster ride at an amusement park: the message queue puts you at the back of the queue, behind all the other people, where you will have to wait for your turn, while the job queue is the fastpass ticket that lets you take another ride right after you finished the previous one.

That's a big difference between Promises (and Async/await, which is built on promises) and plain old asynchronous functions through `setTimeout()` or other platform APIs.

7 Node.js: Event Loop - process.nextTick()

Understanding process.nextTick()

As you try to understand the Node.js event loop, one important part of it is process.nextTick().

Every time the event loop takes a full trip, we call it a tick.

When we pass a function to process.nextTick(), we instruct the engine to invoke this function at the end of the current operation, before the next event loop tick starts:

```
process.nextTick(() => {  
  //do something  
})
```

7 Node.js: Event Loop - `process.nextTick()`

Understanding `process.nextTick()`

The event loop is busy processing the current function code.

When this operation ends, the JS engine runs all the functions passed to `nextTick` calls during that operation.

It's the way we can tell the JS engine to process a function asynchronously (after the current function), but as soon as possible, not queue it.

Calling `setTimeout(() => {}, 0)` will execute the function at the end of next tick, much later than when using `nextTick()` which prioritizes the call and executes it just before the beginning of the next tick.

Use `nextTick()` when you want to make sure that in the next event loop iteration that code is already executed.

7 Node.js: Event Loop - setImmediate()

Understanding setImmediate()

When you want to execute some piece of code asynchronously, but as soon as possible, one option is to use the `setImmediate()` function provided by Node.js:

```
setImmediate(() => {  
  //run something  
})
```

Any function passed as the `setImmediate()` argument is a callback that's executed in the next iteration of the event loop.

How is `setImmediate()` different from `setTimeout(() => {}, 0)` (passing a 0ms timeout), and from `process.nextTick()`?

A function passed to `process.nextTick()` is going to be executed on the current iteration of the event loop, after the current operation ends. This means it will always execute before `setTimeout` and `setImmediate`.

A `setTimeout()` callback with a 0ms delay is very similar to `setImmediate()`. The execution order will depend on various factors, but they will be both run in the next iteration of the event loop.

7 Node.js: Event Loop - setTimeout()

setTimeout()

When writing JavaScript code, you might want to delay the execution of a function.

This is the job of `setTimeout`. You specify a callback function to execute later, and a value expressing how later you want it to run, in milliseconds:

```
setTimeout(() => {  
  // runs after 2 seconds  
}, 2000)  
  
setTimeout(() => {  
  // runs after 50 milliseconds  
}, 50)
```

7 Node.js: Event Loop - setTimeout()

setTimeout()

This syntax defines a new function. You can call whatever other function you want in there, or you can pass an existing function name, and a set of parameters:

```
const myFunction = (firstParam, secondParam) => {  
  // do something  
}  
// runs after 2 seconds  
setTimeout(myFunction, 2000, firstParam, secondParam)
```

setTimeout returns the timer id. This is generally not used, but you can store this id, and clear it if you want to delete this scheduled function execution:

```
const id = setTimeout(() => {  
  // should run after 2 seconds  
}, 2000)  
  
// I changed my mind  
clearTimeout(id)
```

7 Node.js: Event Loop - setTimeout()

Zero delay

If you specify the timeout delay to 0, the callback function will be executed as soon as possible, but after the current function execution:

```
setTimeout(() => {  
  console.log('after ' )  
}, 0)  
  
console.log(' before ')
```

will print before after.

This is especially useful to avoid blocking the CPU on intensive tasks and let other functions be executed while performing a heavy calculation, by queuing functions in the scheduler.

7 Node.js: Event Loop - setInterval()

setInterval()

setInterval is a function similar to setTimeout, with a difference: instead of running the callback function once, it will run it forever, at the specific time interval you specify (in milliseconds):

```
setInterval(() => {  
  // runs every 2 seconds  
}, 2000)
```

The function above runs every 2 seconds unless you tell it to stop, using clearInterval, passing it the interval id that setInterval returned:

```
const id = setInterval(() => { // runs every 2 seconds }, 2000)  
clearInterval(id)
```

7 Node.js: Event Loop - setInterval()

setInterval()

It's common to call `clearInterval` inside the `setInterval` callback function, to let it auto-determine if it should run again or stop. For example this code runs something unless `App.somethingIWait` has the value arrived:

```
const interval = setInterval(() => {  
  if (App.somethingIWait === 'arrived') {  
    clearInterval(interval)  
    return  
  }  
  // otherwise do things  
, 100)
```

8 Node.js: Event emitter

If you worked with JavaScript in the browser, you know how much of the interaction of the user is handled through events: mouse clicks, keyboard button presses, reacting to mouse movements, and so on.

On the backend side, Node.js offers us the option to build a similar system using the events module.

This module, in particular, offers the EventEmitter class, which we'll use to handle our events.

You initialize that using

```
const EventEmitter = require('events')  
const eventEmitter = new EventEmitter()
```

This object exposes, among many others, the on and emit methods.

- emit is used to trigger an event
- on is used to add a callback function that's going to be executed when the event is triggered

8 Node.js: Event emitter

For example, let's create a start event, and as a matter of providing a sample, we react to that by just logging to the console:

```
eventEmitter.on('start', () => {  
  console.log('started')  
})
```

When we run

```
eventEmitter.emit('start')
```

the event handler function is triggered, and we get the console log.

8 Node.js: Event emitter

You can pass arguments to the event handler by passing them as additional arguments to emit():

```
eventEmitter.on('start', number => {  
  console.log(`started ${number}`)  
})  
  
eventEmitter.emit('start', 23)
```

Multiple arguments:

```
eventEmitter.on('start', (start, end) => {  
  console.log(`started from ${start} to ${end}`)  
})  
  
eventEmitter.emit('start', 1, 100)
```

8 Node.js: Event emitter

The EventEmitter object also exposes several other methods to interact with events, like

- `once()`: add a one-time listener
- `removeListener()` / `off()`: remove an event listener from an event
- `removeAllListeners()`: remove all listeners for an event

8 Node.js: Event emitter

For more information on Event Emitters

<https://nodejs.dev/learn/the-nodejs-events-module>

9 Node.js: Asynchronous JS - Introductions

Asynchronicity in Programming Languages

Computers are asynchronous by design.

Asynchronous means that things can happen independently of the main program flow.

In the current consumer computers, every program runs for a specific time slot and then it stops its execution to let another program continue their execution. This thing runs in a cycle so fast that it's impossible to notice. We think our computers run many programs simultaneously, but this is an illusion (except on multiprocessor machines).

Programs internally use interrupts, a signal that's emitted to the processor to gain the attention of the system.

I won't go into the internals of this, but just keep in mind that it's normal for programs to be asynchronous and halt their execution until they need attention, allowing the computer to execute other things in the meantime. When a program is waiting for a response from the network, it cannot halt the processor until the request finishes.

Normally, programming languages are synchronous and some provide a way to manage asynchronicity in the language or through libraries. C, Java, C#, PHP, Go, Ruby, Swift, and Python are all synchronous by default. Some of them handle async by using threads, spawning a new process.

9 Node.js: Asynchronous JS - Introduction

JavaScript

JavaScript is synchronous by default and is single threaded. This means that code cannot create new threads and run in parallel.

But JavaScript was born inside the browser, its main job, in the beginning, was to respond to user actions, like `onClick`, `onMouseOver`, `onChange`, `onSubmit` and so on. How could it do this with a synchronous programming model?

The answer was in its environment. The browser provides a way to do it by providing a set of APIs that can handle this kind of functionality. Node.js introduced a non-blocking I/O environment to extend this concept to file access, network calls and so on.

9 Node.js: Asynchronous JS - Callbacks

Callbacks

You can't know when a user is going to click a button. So, you define an event handler for the click event. This event handler accepts a function, which will be called when the event is triggered:

```
document.getElementById('button').addEventListener('click', () => { //item clicked })
```

This is the so-called callback.

A callback is a simple function that's passed as a value to another function, and will only be executed when the event happens. We can do this because JavaScript has first-class functions, which can be assigned to variables and passed around to other functions (called higher-order functions)

9 Node.js: Asynchronous JS - Promises

Introduction to promises

```
new Promise((resolve, reject) => { resolve("value") } )
```

A promise is commonly defined as a proxy for a value that will eventually become available.

Promises are one way to deal with asynchronous code, without getting stuck in callback hell.

Promises have been part of the language for years (standardized and introduced in ES2015), and have recently become more integrated, with `async` and `await` in ES2017.

Async functions use promises behind the scenes, so understanding how promises work is fundamental to understanding how `async` and `await` work.

9 Node.js: Asynchronous JS - Promises

Introduction to promises

How promises work, in brief

Once a promise has been called, it will start in a pending state. This means that the calling function continues executing, while the promise is pending until it resolves, giving the calling function whatever data was being requested.

The created promise will eventually end in a resolved state, or in a rejected state, calling the respective callback functions (passed to then and catch) upon finishing.

9 Node.js: Asynchronous JS - Promises

Creating a promise

The Promise API exposes a Promise constructor, which you initialize using `new Promise()`:

```
let done = true

const isItDoneYet = new Promise((resolve, reject) => {
  if (done) {
    const workDone = 'Here is the thing I built'
    resolve(workDone)
  } else {
    const why = 'Still working on something else'
    reject(why)
  }
})
```

9 Node.js: Asynchronous JS - Promises

Creating a promise

As you can see, the promise checks the `done` global constant, and if that's true, the promise goes to a resolved state (since the `resolve` callback was called); otherwise, the `reject` callback is executed, putting the promise in a rejected state. (If none of these functions is called in the execution path, the promise will remain in a pending state)

Using `resolve` and `reject`, we can communicate back to the caller what the resulting promise state was, and what to do with it. In the above case we just returned a string, but it could be an object, or null as well. Because we've created the promise in the above snippet, it has already started executing.

9 Node.js: Asynchronous JS - Promises

Creating a promise

A more common example you may come across is a technique called Promisifying. This technique is a way to be able to use a classic JavaScript function that takes a callback, and have it return a promise:

```
const fs = require('fs')

const getFile = (fileName) => {
  return new Promise((resolve, reject) => {
    fs.readFile(fileName, (err, data) => {
      if (err) {
        reject(err) // calling `reject` will cause the promise to fail with or without the error passed as an argument
        return      // and we don't want to go any further
      }
      resolve(data)
    })
  })
}

getFile('/etc/passwd')
  .then(data => console.log(data))
  .catch(err => console.error(err))
```

9 Node.js: Asynchronous JS - Promises

Consuming a promise

In the last section, we introduced how a promise is created. Now let's see how the promise can be consumed or used.

```
const isItDoneYet = new Promise(/* ... as above ... */)
//...

const checkIfItsDone = () => {
  isItDoneYet
    .then(ok => {
      console.log(ok)
    })
    .catch(err => {
      console.error(err)
    })
}
```

9 Node.js: Asynchronous JS - Promises

Chaining promises

A promise can be returned to another promise, creating a chain of promises.

A great example of chaining promises is the Fetch API, which we can use to get a resource and queue a chain of promises to execute when the resource is fetched.

The Fetch API is a promise-based mechanism, and calling `fetch()` is equivalent to defining our own promise using `new Promise()`.

9 Node.js: Asynchronous JS - Promises

Chaining promises

Example of chaining promises

```
const status = response => {
  if (response.status >= 200 && response.status < 300) {
    return Promise.resolve(response)
  }
  return Promise.reject(new Error(response.statusText))
}

const json = response => response.json()

fetch('/todos.json')
  .then(status)    // note that the `status` function is actual
  .then(json)      // likewise, the only difference here is tha
  .then(data => { // ... which is why `data` shows up here as
    console.log('Request succeeded with JSON response', data)
  })
  .catch(error => {
    console.log('Request failed', error)
  })
})
```

9 Node.js: Asynchronous JS - Promises

Orchestrating promises

Promise.all()

If you need to synchronize different promises, Promise.all() helps you define a list of promises, and execute something when they are all resolved.

```
const f1 = fetch('/something.json')
const f2 = fetch('/something2.json')

Promise.all([f1, f2])
  .then(res => {
    console.log('Array of results', res)
  })
  .catch(err => {
    console.error(err)
  })
```

9 Node.js: Asynchronous JS - Promises

Orchestrating promises

Promise.race()

Promise.race() runs when the first of the promises you pass to it resolves, and it runs the attached callback just once, with the result of the first promise resolved.

```
const first = new Promise((resolve, reject) => {
  setTimeout(resolve, 500, 'first')
})
const second = new Promise((resolve, reject) => {
  setTimeout(resolve, 100, 'second')
})

Promise.race([first, second]).then(result => {
  console.log(result) // second
})
```


9 Node.js: Asynchronous JS - Async/Await

Introduction to Async/Await

JavaScript evolved in a very short time from callbacks to promises (ES2015), and since ES2017 asynchronous JavaScript is even simpler with the `async/await` syntax.

Async functions are a combination of promises and generators, and basically, they are a higher level abstraction over promises. Let me repeat: `async/await` is built on promises.

9 Node.js: Asynchronous JS - Async/Await

Why were `async/await` introduced?

They reduce the boilerplate around promises, and the "don't break the chain" limitation of chaining promises.

When Promises were introduced in ES2015, they were meant to solve a problem with asynchronous code, and they did, but over the 2 years that separated ES2015 and ES2017, it was clear that promises could not be the final solution.

Promises were introduced to solve the famous callback hell problem, but they introduced complexity on their own, and syntax complexity.

They were good primitives around which a better syntax could be exposed to the developers, so when the time was right we got `async` functions.

They make the code look like it's synchronous, but it's asynchronous and non-blocking behind the scenes.

9 Node.js: Asynchronous JS - Async/Await

How it works

An async function returns a promise, like in this example:

```
const doSomethingAsync = () => {  
  return new Promise(resolve => {  
    setTimeout(() => resolve('I did something'), 3000)  
  })  
}
```

When you want to call this function you prepend `await`, and the calling code will stop until the promise is resolved or rejected. One caveat: the client function must be defined as `async`. Here's an example:

```
const doSomething = async () => {  
  console.log(await doSomethingAsync())  
}
```

9 Node.js: Asynchronous JS - Async/Await

Promise all the things

Prepending the `async` keyword to any function means that the function will return a promise.

Even if it's not doing so explicitly, it will internally make it return a promise.

This is why this code is valid:

```
const aFunction = async () => {  
  return 'test'  
}  
  
aFunction().then(alert) // This will alert 'test'
```

and it's the same as:

```
const aFunction = () => {  
  return Promise.resolve('test')  
}  
  
aFunction().then(alert) // This will alert 'test'
```

9 Node.js: Asynchronous JS - Async/Await

The code is much simpler to read

As you can see in the example above, our code looks very simple. Compare it to code using plain promises, with chaining and callback functions.

And this is a very simple example, the major benefits will arise when the code is much more complex.

.

9 Node.js: Asynchronous JS - Async/Await

Multiple async functions in series

Async functions can be chained very easily, and the syntax is much more readable than with plain promises:

.

11 Node.js: Streams - What are streams?

Streams are one of the fundamental concepts that power Node.js applications.

They are a way to handle reading/writing files, network communications, or any kind of end-to-end information exchange in an efficient way.

Streams are not a concept unique to Node.js. They were introduced in the Unix operating system decades ago, and programs can interact with each other passing streams through the pipe operator (`|`).

For example, in the traditional way, when you tell the program to read a file, the file is read into memory, from start to finish, and then you process it.

Using streams you read it piece by piece, processing its content without keeping it all in memory.

The Node.js stream module provides the foundation upon which all streaming APIs are built. All streams are instances of `EventEmitter`

11 Node.js: Streams - Why streams?

Streams basically provide two major advantages over using other data handling methods:

- Memory efficiency: you don't need to load large amounts of data in memory before you are able to process it
- Time efficiency: it takes way less time to start processing data, since you can start processing as soon as you have it, rather than waiting till the whole data payload is available

11 Node.js: Streams - An example of a stream

A typical example is reading files from a disk.

Using the Node.js fs module, you can read a file, and serve it over HTTP when a new connection is established to your HTTP server:

```
const http = require('http')
const fs = require('fs')

const server = http.createServer(function(req, res) {
  fs.readFile(__dirname + '/data.txt', (err, data) => {
    res.end(data)
  })
})
server.listen(3000)
```

`readFile()` reads the full contents of the file, and invokes the callback function when it's done. `res.end(data)` in the callback will return the file contents to the HTTP client.

11 Node.js: Streams - An example of a stream

If the file is big, the operation will take quite a bit of time. Here is the same thing written using streams:

```
const http = require('http')
const fs = require('fs')

const server = http.createServer((req, res) => {
  const stream = fs.createReadStream(__dirname + '/data.txt')
  stream.pipe(res)
})
server.listen(3000)
```

Instead of waiting until the file is fully read, we start streaming it to the HTTP client as soon as we have a chunk of data ready to be sent.

11 Node.js: Streams - pipe()

The above example uses the line `stream.pipe(res)`: the `pipe()` method is called on the file stream.

What does this code do? It takes the source, and pipes it into a destination.

You call it on the source stream, so in this case, the file stream is piped to the HTTP response.

The return value of the `pipe()` method is the destination stream, which is a very convenient thing that lets us chain multiple `pipe()` calls, like this:

```
src.pipe(dest1).pipe(dest2)
```

This construct is the same as doing

```
src.pipe(dest1)  
dest1.pipe(dest2)
```

11 Node.js: Streams - Streams-powered Node.js APIs

Due to their advantages, many Node.js core modules provide native stream handling capabilities, most notably:

- `process.stdin` returns a stream connected to `stdin`
- `process.stdout` returns a stream connected to `stdout`
- `process.stderr` returns a stream connected to `stderr`
- `fs.createReadStream()` creates a readable stream to a file
- `fs.createWriteStream()` creates a writable stream to a file
- `net.connect()` initiates a stream-based connection
- `http.request()` returns an instance of the `http.ClientRequest` class, which is a writable stream
- `zlib.createGzip()` compress data using `gzip` (a compression algorithm) into a stream
- `zlib.createGunzip()` decompress a `gzip` stream.
- `zlib.createDeflate()` compress data using `deflate` (a compression algorithm) into a stream
- `zlib.createInflate()` decompress a `deflate` stream

11 Node.js: Streams - Different types of streams

There are four classes of streams:

- Readable: a stream you can pipe from, but not pipe into (you can receive data, but not send data to it). When you push data into a readable stream, it is buffered, until a consumer starts to read the data.
- Writable: a stream you can pipe into, but not pipe from (you can send data, but not receive from it)
- Duplex: a stream you can both pipe into and pipe from, basically a combination of a Readable and Writable stream
- Transform: a Transform stream is similar to a Duplex, but the output is a transform of its input

11 Node.js: Streams - How to create a readable stream

We get the Readable stream from the stream module, and we initialize it and implement the `readable._read()` method.

First create a stream object:

```
const Stream = require('stream')
const readableStream = new Stream.Readable()
```

then implement `_read`:

```
readableStream._read = () => {}
```

You can also implement `_read` using the `read` option:

```
const readableStream = new Stream.Readable({
  read() {}
})
```

Now that the stream is initialized, we can send data to it:

```
readableStream.push('hi!')
readableStream.push('ho!')
```

11 Node.js: Streams - How to create a writable stream

To create a writable stream we extend the base `Writable` object, and we implement its `_write()` method.

First create a stream object:

```
const Stream = require('stream')
const writableStream = new Stream.Writable()
```

then implement `_write`:

```
writableStream._write = (chunk, encoding, next) => {
  console.log(chunk.toString())
  next()
}
```

You can now pipe a readable stream in:

```
process.stdin.pipe(writableStream)
```

11 Node.js: Streams - Get data from a readable stream

How do we read data from a readable stream? Using a writable stream:

```
const Stream = require('stream')

const readableStream = new Stream.Readable({
  read() {}
})

const writableStream = new Stream.Writable()

writableStream._write = (chunk, encoding, next) => {
  console.log(chunk.toString())
  next()
}

readableStream.pipe(writableStream)

readableStream.push('hi!')
readableStream.push('ho!')
```


11 Node.js: Streams - Get data from a readable stream

You can also consume a readable stream directly, using the `readable` event:

```
readableStream.on('readable', () => {  
  console.log(readableStream.read())  
})
```

11 Node.js: Streams - Send data to a writable stream

Using the stream `write()` method:

```
writableStream.write('hey!\n')
```

11 Node.js: Streams - End a writable stream

Use the `end()` method:

```
const Stream = require('stream')

const readableStream = new Stream.Readable({
  read() {}
})
const writableStream = new Stream.Writable()

writableStream._write = (chunk, encoding, next) => {
  console.log(chunk.toString())
  next()
}

readableStream.pipe(writableStream)

readableStream.push('hi!')
readableStream.push('ho!')

writableStream.end()
```

11 Node.js: Streams - Create a transform stream

We get the Transform stream from the `stream` module, and we initialize it and implement the `transform._transform()` method.

First create a transform stream object:

```
const { Transform } = require('stream')
const TransformStream = new Transform;
```

then implement `_transform`:

```
TransformStream._transform = (chunk, encoding, callback) => {
  console.log(chunk.toString().toUpperCase());
  callback();
}
```

Pipe readable stream:

```
process.stdin.pipe(TransformStream);
```

10 Node- Buffers - What is a buffer?

A buffer is an area of memory. JavaScript developers are not familiar with this concept, much less than C, C++ or Go developers (or any programmer that uses a system programming language), which interact with memory every day.

It represents a fixed-size chunk of memory (can't be resized) allocated outside of the V8 JavaScript engine.

You can think of a buffer like an array of integers, which each represent a byte of data.

It is implemented by the Node.js Buffer class.

10 Node- Buffers - Why do we need a buffer?

Buffers were introduced to help developers deal with binary data, in an ecosystem that traditionally only dealt with strings rather than binaries.

Buffers are deeply linked with streams. When a stream processor receives data faster than it can digest, it puts the data in a buffer.

A simple visualization of a buffer is when you are watching a YouTube video and the red line goes beyond your visualization point: you are downloading data faster than you're viewing it, and your browser buffers it.

10 Node- Buffers - How to create a buffer?

A buffer is created using the `Buffer.from()`, `Buffer.alloc()`, and `Buffer.allocUnsafe()` methods.

```
const buf = Buffer.from('Hey!')
```

- `Buffer.from(array)`
- `Buffer.from(arrayBuffer[, byteOffset[, length]])`
- `Buffer.from(buffer)`
- `Buffer.from(string[, encoding])`

10 Node- Buffers - How to create a buffer?

You can also just initialize the buffer passing the size. This creates a 1KB buffer:

```
const buf = Buffer.alloc(1024)
//or
const buf = Buffer.allocUnsafe(1024)
```

While both `alloc` and `allocUnsafe` allocate a Buffer of the specified size in bytes, the Buffer created by `alloc` will be initialized with zeroes and the one created by `allocUnsafe` will be uninitialized. This means that while `allocUnsafe` would be quite fast in comparison to `alloc`, the allocated segment of memory may contain old data which could potentially be sensitive.

10 Node- Buffers - Using a buffer

Access the content of a buffer

A buffer, being an array of bytes, can be accessed like an array:

```
const buf = Buffer.from('Hey!')  
console.log(buf[0]) //72  
console.log(buf[1]) //101  
console.log(buf[2]) //121
```

Those numbers are the Unicode Code that identifies the character in the buffer position (H => 72, e => 101, y => 121)

```
console.log(buf.toString())
```

You can print the full content of the buffer using the toString() method:

10 Node- Buffers - Using a buffer

Get the length of a buffer

Use the length property:

```
const buf = Buffer.from('Hey!')  
console.log(buf.length)
```

Iterate over the contents of a buffer

```
const buf = Buffer.from('Hey!')  
for (const item of buf) {  
  console.log(item) //72 101 121 33  
}
```

10 Node- Buffers - Using a buffer

Changing the content of a buffer

You can write to a buffer a whole string of data by using the write() method:

```
const buf = Buffer.alloc(4)
buf.write('Hey!')
```

Just like you can access a buffer with an array syntax, you can also set the contents of the buffer in the same way:

```
const buf = Buffer.from('Hey!')
buf[1] = 111 //o
console.log(buf.toString()) //Hoy!
```

10 Node- Buffers - Using a buffer

Copy a buffer

Copying a buffer is possible using the `copy()` method:

```
const buf = Buffer.from('Hey!')
let bufcopy = Buffer.alloc(4) //allocate 4 bytes
buf.copy(bufcopy)
```

By default you copy the whole buffer. 3 more parameters let you define the target buffer starting position to copy to, the source buffer starting position to copy from, and the new buffer length:

```
const buf = Buffer.from('Hey!')
let bufcopy = Buffer.alloc(2) //allocate 2 bytes
buf.copy(bufcopy, 0, 0, 2)
bufcopy.toString() //'He'
```

10 Node- Buffers - Using a buffer

Slice a buffer

If you want to create a partial visualization of a buffer, you can create a slice. A slice is not a copy: the original buffer is still the source of truth. If that changes, your slice changes.

Use the `slice()` method to create it. The first parameter is the starting position, and you can specify an optional second parameter with the end position:

```
const buf = Buffer.from('Hey!')
buf.slice(0).toString() //Hey!
const slice = buf.slice(0, 2)
console.log(slice.toString()) //He
buf[1] = 111 //o
console.log(slice.toString()) //Ho
```

12 Node: Error Handling - Exceptions

Errors in Node.js are handled through exceptions.

Creating exceptions

An exception is created using the throw keyword:

```
throw value
```

As soon as JavaScript executes this line, the normal program flow is halted and the control is held back to the nearest exception handler.

Usually in client-side code value can be any JavaScript value including a string, a number or an object.

In Node.js, we don't throw strings, we just throw Error objects.

12 Node: Error Handling - Error Objects

An error object is an object that is either an instance of the Error object, or extends the Error class, provided in the Error core module:

```
throw new Error('Ran out of coffee')
```

or

```
class NotEnoughCoffeeError extends Error {  
  //...  
}  
throw new NotEnoughCoffeeError()
```

12 Node: Error Handling - Handling Exceptions

An exception handler is a try/catch statement.

Any exception raised in the lines of code included in the try block is handled in the corresponding catch block:

```
try {  
    //lines of code  
} catch (e) {}
```

e in this example is the exception value.

You can add multiple handlers, that can catch different kinds of errors.

12 Node: Error Handling - Catching uncaught exceptions

If an uncaught exception gets thrown during the execution of your program, your program will crash.

To solve this, you listen for the `uncaughtException` event on the process object:

```
process.on('uncaughtException', err => {  
  console.error('There was an uncaught error', err)  
  process.exit(1) //mandatory (as per the Node.js docs)  
})
```

You don't need to import the process core module for this, as it's automatically injected.

12 Node: Error Handling - Exceptions with promises

Using promises you can chain different operations, and handle errors at the end:

```
doSomething1()  
  .then(doSomething2)  
  .then(doSomething3)  
  .catch(err => console.error(err))
```

You can handle errors in each of the functions you call (doSomethingX), and inside the error handler throw a new error, that's going to call the outside catch handler:

```
const doSomething1 = () => {  
  //...  
  try {  
    //...  
  } catch (err) {  
    //... handle it locally  
    throw new Error(err.message)  
  }  
  //...  
}
```

12 Node: Error Handling - Exceptions with promises

To be able to handle errors locally without handling them in the function we call, we can break the chain you can create a function in each then() and process the exception:

```
doSomething1()
  .then(() => {
    return doSomething2().catch(err => {
      //handle error
      throw err //break the chain!
    })
  })
  .then(() => {
    return doSomething3().catch(err => {
      //handle error
      throw err //break the chain!
    })
  })
  .catch(err => console.error(err))
```

12 Node: Error Handling - Error handling with async/await

Using async/await, you still need to catch errors, and you do it this way:

```
async function someFunction() {  
  try {  
    await someOtherFunction()  
  } catch (err) {  
    console.error(err.message)  
  }  
}
```

The End.

COMING SOON... TO A MOODLE NEAR YOU!

Labs

Web Server in Node & npm

Homework

TBD